

p0834r0

Lifting overload sets into objects

Published Proposal, 16 October 2017

This version:

<http://wg21.link/P0834R0>

Author:

[Michał Dominiak](#) (Nokia Networks)

Audience:

EWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Passing overload sets around made easy

Table of Contents

1	Introduction
1.1	Motivation
1.2	Previous standardization attempts
1.3	Conventions
2	Core proposal
2.1	Passing overloaded functions
2.2	Passing operators
2.3	Member access
2.4	Members vs member functions

2.5	Impact on the standard
3	Things to consider
3.1	Forced materialization
3.2	Further generalization of the feature
4	Acknowledgements

§ 1. Introduction

§ 1.1. Motivation

Passing functions around in C++ is a common occurrence, however it's an extremely difficult thing to do when you want to pass an entire overload set into another function. In some cases, you can simply pass a function pointer - but that can't be relied upon for, for example, standard functions, as standard functions have no guaranteed signatures.

One way that somewhat captures the intent of "call a function called `foo`" is:

```
auto overload_set = [](auto &&... args)
    -> decltype(foo(std::forward<decltype(args)>(args)...))
{
    return foo(std::forward<decltype(args)>(args)...);
};
```

This can be simplified with a macro, but I believe that a dedicated language feature that can handle more cases than just named functions is a good idea; other things that the proposed syntax can be extended to is operators or member access.

A more concrete motivating example has been given in P0119:

```

template<typename T>
T twice(T x)
{
    return x + x;
}

template<typename It>
void f(It first, It second)
{
    std::transform(first, second, first, twice);
}

```

This currently doesn't work; and this paper doesn't propose to make it work. Instead, it proposes we introduce the following syntax:

```

template<typename It>
void f(It first, It second)
{
    std::transform(first, second, first, []twice);
}

```

§ 1.2. Previous standardization attempts

The original proposal is [N3617](#). Later, this functionality has been proposed in [P0119](#), which attempted to remove the syntax requirement for the lambda introducers; this caused problems that were discovered during the discussion in Oulu. The problems included, but were not limited to, ODR issues; more information can be found in the discussion notes.

This proposal goes back to the syntax using lambda introducers, but with a different take on member accessors.

The original paper also introduced a notion of [LIFTED_INVOKE](#), which this paper doesn't attempt to do. I intend to go with this paper, as a design paper, through EWG, and then come back again to apply any comments and actually provide wording. If the recommendation is to invent something similar to [INVOKE](#), I will do that.

§ 1.3. Conventions

To make the code snippets more readable, this paper assumes the following macro:

```
#define FWD(...) std::forward<decltype(__VA_ARGS__)>(__VA_ARGS__)
```

§ 2. Core proposal

§ 2.1. Passing overloaded functions

The core of the proposal is the ability to pass functions "by name", that is - to pass an object representing an entire overload set around. The basic transformation would be from

```
[]<id-expression>
```

into

```
[](auto &&... args)
    noexcept(noexcept(<id-expression>(FWD(args)...)))
    -> decltype(<id-expression>(FWD(args)...))
{
    return <id-expression>(FWD(args)...);
}
```

Notice that this proposes the syntax to allow `id-expressions`; this means that expressions like `[]std::addressof` would be valid, which solves the problem of "can't rely on specific signatures in `std::`").

§ 2.2. Passing operators

The second proposed form is

```
[]@
```

where `@` is an operator. This would be transformed into (assuming `@` is binary)

```

[](auto && arg1, auto && arg2)
    noexcept(noexcept(FWD(arg1) @ FWD(arg2)))
    -> decltype(FWD(arg1) @ FWD(arg2))
{
    return FWD(arg1) @ FWD(arg2);
}

```

This is arguably already provided by the family of `std::less<>`, but `[]<` is much shorter than `std::less<>()`.

For unary operators:

```

[](auto && arg)
    noexcept(@ FWD(arg))
    -> decltype(@ FWD(arg))
{
    return @ FWD(arg);
}

```

For operators that can be both unary and binary, like `+`, `[]@` would create a function object that can be called in both ways.

There's four operators that are somewhat problematic here, as they don't follow the general pattern.

1. The call operator object `[]()` - I do not know whether this can be made unambiguous enough, but even if it can - I'm not sure if we want to introduce a potential source of confusion (against `[](){}).`
2. The indexing operator object `[][]` - this looks funny, but should be workable, and the semantics are pretty obvious.
3. The `[]++` and `[]--` operators. P0119 proposes that they should always only mean pre-increment and pre-decrement, and I agree with that judgement, but I wouldn't be too irritated if we just ignored them and decided to not support the silliness.

One solution for (1) and (2) would be to require the syntax to be `[]operator()` or `[]operator[]`, but that is quite verbose. An argument can be made for allowing this more verbose form for all operators, that is to make `[]@` equivalent to `[]operator@` in all cases where it is allowed.

§ 2.3. Member access

The final form proposed is related to accessing members of structures and classes. Currently the only way to pass a *thing* naming a specific member is to pass a member function pointer or a member pointer around. Those are unwieldy, and are not generic, that is point to a specific member of a specific class, generally by an offset or something similar. But more frequently what we need is a tool to actually allow us to *name* a member, and have that named member reference be generic, i.e. usable with different types.

The proposed syntax differs from the original paper, and uses a disambiguator to clearly state whether a lifted object is meant to be a free function or a member; that is:

```
[].<identifier>
```

would be transformed into

```
[](auto && object) noexcept  
-> decltype((FWD(object).<identifier>)) // intentional double parentheses  
{  
    return FWD(object).<identifier>;  
}
```

This would again allow us to make this example from the original paper (only slightly modified) compile:

```
struct user{  
    int id;  
    std::string name;  
};  
  
void f()  
{  
    std::vector<user> users{ {4, "Bjarne"}, {1, "Herb"}, {3, "Scott"}, {0,  
    // ordered_by implementation left as an exercise to the reader  
    std::sort(users.begin(), users.end(), ordered_by([].id));  
    // [(0, "STL"), (1, "Herb"), (2, "Andrei"), (3, "Scott"), (4, "Bjarne")  
    std::sort(users.begin(), users.end(), ordered_by([].name));  
    // [(2, "Andrei"), (4, "Bjarne"), (1, "Herb"), (3, "Scott"), (0, "Steph  
}
```

§ 2.4. Members vs member functions

When `[].<identifier>` refers to a data member, there's no easy way to actually specify "I want to name a member function and invoke it", instead of "I want to name a data member". We could invent a weird and outlandish syntax like `[].( )<identifier>`, but despite that being quite short, it doesn't look very good, and the gut feeling is that it's too close to Haskell's variety of operators for the sanity of C++ programmers.

One way to approach this, proposed in a discussion by Tomasz Kamiński, is to always invoke when possible, and return a value if not. We could also return when not a function and call otherwise. Personally I consider this troublesome; the meaning of code can silently change without detecting something that should be an error, when the lifted accessor does too much magic. Consider

```
struct foo
{
    int a;
};
```

If we call `[] .a` with an instance of `foo`, we will be given a reference to the member. However, if someone changes the definition to the following:

```
struct foo
{
    int a() const;
};
```

then `[] .a` will continue to work on instances of `foo`, even though the meaning changed. However, all other places that just access `foo::a` directly, not via a lifting expression, will stop compiling. On the flip side, if we have the second definition, and someone changes it to

```
struct foo
{
    std::function<int ()> a;
};
```

then, depending on whether we chose to call if we can, or to return if we can, we will only have the ability to lift a call to `foo::a`, or to lift an access of `foo::a`; we won't have both; and despite the change being source-compatible everywhere else, it can be an incompatible change in the presence of lifted accessors.

If we decide to have a dedicated syntax for lifting member accesses, and a dedicated syntax for lifting member calls, then both those operations are always possible to explicitly express, the meaning of code won't silently change without causing a compilation error, and will keep compiling through source-compatible changes.

§ 2.5. Impact on the standard

The proposed syntax is currently invalid, so there's no impact on other parts of the standard or other features.

Abbreviated lambdas may potentially have impact on this feature, if they get accepted, by simplifying the specification of what lifted expressions are transformed into. (Unless we want to specify this differently, to avoid the forced materialization issue mentioned below.)

§ 3. Things to consider

§ 3.1. Forced materialization

In an email discussion, Tomasz Kamiński mentioned one problem with the naive transformations presented in this paper as the workaround and as the equivalent of `[]foo`: they will force a materialization of an object potentially too early and potentially invoke an additional move constructor, when used over a function directly.

To spell the issue out more directly in code: if we currently have a following function:

```
void f(std::string);
```

The invocation in the form of `f("abc"s)` will directly initialize argument. However if the function is wrapped in a "perfectly" forwarding lambda, or lift it by writing `auto lf = []f;`, the `lf("abc"s)` invocation will perform an additional move over `f("abc"s)`.

This is a general problem with "perfect" forwarding an materialization, but we might be able to somehow solve it for this particular case, by making the calls through `[]f` equivalent to just calling `f` directly.

This might be one reason to reinvent `LIFTED_INVOKE` from the original paper, but further investigation into viability and exact semantics would be required.

§ 3.2. Further generalization of the feature

Barry Rezvin mentioned that this feature feels weirdly specific, and doesn't by itself support things like adding arguments into the call. I understand the sentiment, and perhaps there's a way to make that work, but I'm quite convinced that it doesn't belong to this particular feature; it might be a pretty good extension though, if we figure out the details.

§ 4. Acknowledgements

I'd like to thank Philipp Juschka for the original proposal, Andrew Sutton for picking it up later on, and again Andrew, this time together with Tomasz Kamiński, for further input and comments.